

RAG & LangGraph — Study Notes

1. Foundation & Context

These notes cover the foundation for **LangChain**, with a focus on **LangGraph for Python** and the **RAG (Retrieval Augmented Generation)** architecture.

What is RAG?

RAG stands for **Retrieval Augmented Generation**. It is a technique that enhances Large Language Models (LLMs) by allowing them to retrieve relevant information from an external knowledge base before generating a response.

LLM Limitations (Why RAG?)

Standard LLMs suffer from three core limitations:

- **Knowledge Cut-off** — The model's training data has a fixed end date; it cannot know about recent events.
- **Context Limit** — LLMs can only process a limited amount of text at a time (context window).
- **Proprietary Data** — LLMs have no access to private, organization-specific, or confidential data.

RAG Pipeline

The RAG pipeline consists of two parallel flows:

Indexing Flow (Offline):

Data → Chunks → Embeddings → Vector DB

Query Flow (Online):

Query + Prompt → (Chunks via Vector DB) → LLM → Answer

RAG Principles

2. Chunking

Chunking splits raw data into smaller pieces based on a **token limit**. It is a critical step because:

- Entire data cannot be stored at once — this affects retrieval quality.
- If all content is in a single vector, the DB must search everything for every user query, reducing retrieval quality.
- When each topic is in its own separate chunk, retrieval becomes very fast and accurate.

Types of Chunking

1. Fixed Chunking

A fixed token limit (e.g., `chunk_size = 500`) is applied uniformly. Every chunk stores exactly 500 words/tokens simultaneously.

Problem: Sentences may be cut mid-way. Chunk 1 contains half a sentence and Chunk 2 contains the other half, which reduces retrieval quality.

2. Recursive Chunking

Stores meaningful paragraphs (complete content) within the token limit. Uses a hierarchy of separators:

```
separators = ["\n\n", "\n", " ", ""]
```

Example with `chunk_size = 1000`:

Para 1 (600 tokens) → **Chunk 1:** Para 1 + Para 2

Para 2 (400 tokens) → **Chunk 2:** Para 3 only

Para 3 (800 tokens) → **Chunk 3:** Para 4

Para 4 (300 tokens) → **Chunk 4:** Para 5 exceeds limit → next separator

The logic: separate paragraphs with small paras or store based on lines. If it crosses limits, store separated by spaces or words.

Goal: Meaningful chunks, not full chunks.

Chunk Overlap & Embeddings

3. Chunk Overlap

Context can be missed near chunk boundaries. To solve this, chunks are made to overlap:

Example:

```
Chunk 1 → tokens 1-500
```

```
Chunk 2 → tokens 400-900
```

```
Tokens 400-500 → Overlapped region
```

This ensures boundary data is completely stored in at least one of the chunks.

4. Embeddings

An embedding is a **list of numbers (a vector)** that represents the semantic meaning of text. Similar concepts are placed close together in vector space.

Examples:

```
cat = [0.2, 0.8, -0.1]
```

```
dog = [0.25, 0.75, -0.15]
```

```
car = [-0.9, 0.01, 0.8]
```

In a 2D model, dimensions might represent concepts like *animalness*, *vehicleness*, etc. Real embeddings have hundreds or thousands of dimensions.

How Does Embedding Happen?

Text passes through the following pipeline:

```
Text → Tokenizer → Tokens → Transformer Layers → Vector
```

Cosine Similarity

Cosine similarity measures how close two vectors are by checking their direction. If two vectors point in the same direction, similarity is high.

```
Similarity = cosine(query_vector, stored_vector)
```

Example:

```
Query = [0.8, 0.17]
```

```
Vector DB: Chunk 1: [0.99, 0.2] → Similarity: 0.99 ✓
```

```
Chunk 2: [0.4, -0.1] → Similarity: 0.24
```

Embeddings are useful because of **semantic search** — finding meaning, not just keywords.

Vector DB, Data Collection & Retrieval

5. Full Retrieval Flow

When a user submits a query, the system follows this pipeline:

```
User Query → Embedding Model → Query Vector → Vector DB → Cosine Similarity → Top-K Chunks → LLM → Answer
```

6. Vector DB

A Vector DB uses **ANN (Approximate Nearest Neighbour)** search. Instead of searching all stored vectors exhaustively, it efficiently finds the nearest neighbour to the query vector.

7. Data Collection Phase

Data Cleaning steps include:

- Remove whitespace / extra spaces
- Unicode removal
- OCR (for scanned documents)
- Remove headers and footers
- Rule-based filtering
- De-duplication
- Language detection

Key principle: Garbage In → Garbage Out

8. Choosing the Right Storage Strategy

- **Structured data** → Use SQL or a Knowledge Graph (no embeddings needed).
- **Unstructured data** (PDFs, wiki, policies, documents) → Use RAG + Embeddings + Vector DB.
- **Relationship data** (e.g., John → works at → TNG → handled by → Sarah → manages Team) → Use **RAG Graph** or Knowledge Graph (no embeddings needed).

Vector DB Search Algorithms

Multiple algorithms exist for efficient searching, such as **HNSW**. These algorithms return the closest chunks ranked 1, 2, 3...

The most important metrics here are **Precision** and **Recall**.

Precision, Recall & Re-Ranking

9. Precision vs. Recall

Precision — Finding the answer in very few chunks; retrieving the best answer from ranking. High precision means fewer but more relevant results.

Recall — Finding and retrieving *all* the relevant chunks. High recall means no relevant result is missed.

The Trade-off

- If the system returns only the **top 2 chunks** → Precision is HIGH, Recall is LOW.
- If the system returns the **top 20 chunks** → Information may be extra/noisy, Precision will be very LOW.

10. Re-Ranking (Solution to the Trade-off)

Re-Ranking solves the precision–recall trade-off in two steps:

Step 1 — High Recall: Get many relevant chunks from the vector DB using HNSW (recall ↑).

Step 2 — High Precision: Minimize chunks by checking whether each chunk actually answers the question asked (precision ↑, recall ↓ slightly).

How Re-Ranking Works

Re-Ranking feeds **both the question and the chunk** into a Transformer model:

- The Transformer can easily analyze word relationships and identify whether the chunk is relevant to the question.
- It returns precise, ranked results.

Summary

RAG overcomes LLM limitations by retrieving relevant context from a vector database at query time. The full pipeline involves data cleaning, recursive chunking with overlap, embedding generation, ANN-based vector search, and re-ranking to deliver precise and complete answers.

LangGraph for Python provides the orchestration layer to implement these agentic RAG workflows.